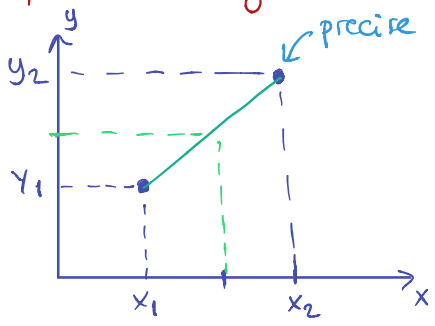
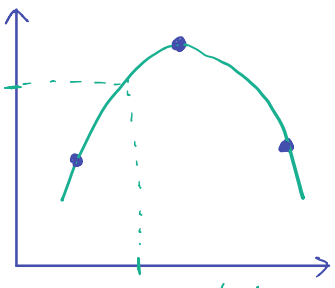
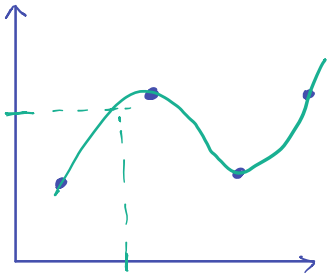
Chapter 17 – Polynomial Interpolation



line : 1st order polynomial



A parabola



Goal : to estimate intermediate data pts between precise data points.

$(n-1)^{th}$ order polynomial

$$f(x) = a_1 + a_2 x + a_3 x^2 + \cdots + a_n x^{n-1}$$

Polynomial interpolation

n data pts are needed to determine n coefficients.

Ex Data: $x_1 = 300$, $f(x_1) = 0.616$

$x_2 = 400$, $f(x_2) = 0.525$

$x_3 = 500$, $f(x_3) = 0.457$

want to determine the coefficients of the parabola $f(x) = p_1 x^2 + p_2 x + p_3$ that passes through the points above

Sol^n : Put each pt. into the equation:

$$\left.\begin{aligned} 0.616 &= p_1(300)^2 + p_2(300) + p_3 \\ 0.525 &= p_1(400)^2 + p_2(400) + p_3 \\ 0.457 &= p_1(500)^2 + p_2(500) + p_3 \end{aligned}\right\} \text{3 eq}^n\text{s, 3 unknowns}$$

in matrix form

$$\begin{bmatrix} 90000 & 300 & 1 \\ 160000 & 400 & 1 \\ 250000 & 500 & 1 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} = \begin{bmatrix} 0.616 \\ 0.525 \\ 0.457 \end{bmatrix} \Rightarrow$$

Solve:

$p_1 = 0.00\,000115$

$p_2 = -0.001715$

$p_3 = 1.027$

$$f(x) = 0.00000115x^2 - 0.001715x + 1.027$$

Evaluate $f(x)$ at an intermediate value:

$$f(350) = 0.567625$$

Drawbacks of above approach:

Coefficient matrix:

$$\underbrace{\begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \end{bmatrix}}_{\text{very ill-conditioned}} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} = \begin{bmatrix} f(x_1) \\ f(x_2) \\ f(x_3) \end{bmatrix}$$

$\text{cond}(A) = 5.89 \times 10^6$ (for prev. exercise)

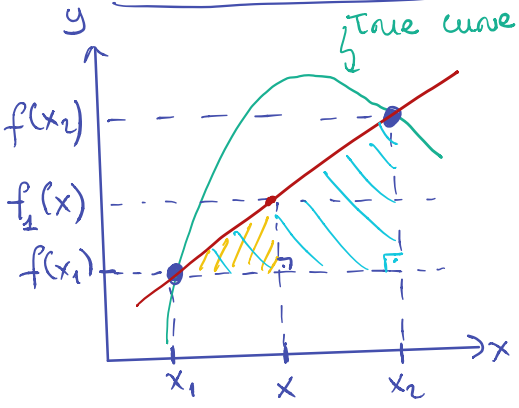
Means: 6 digits of the solution is questionable

* Ill-conditioning becomes worse if # eq^{n}s are bigger
* Approaches exist, which do not have this drawback

Newton Interpolating Polynomial

An alternative form of expressing an interpolating polynomial

Linear Interpolation: connect 2 pts with a straight line



using similar triangles (yellow & blue)

1st order: $$\frac{f_1(x) - f(x_1)}{x - x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$$

slope of line

Arrange:

$$f_1(x) = f(x_1) + \frac{f(x_2) - f(x_1)}{x_2 - x_1}(x - x_1)$$

Newton interpolation formula

* Given 2 pts: $(x_1, f(x_1))$, $(x_2, f(x_2))$

* The smaller the interval btwn data pts, the better the approximation.

EX Estimate $\ln 2$ using linear interpolation

(a) Interpolate btwn $\ln 1$ and $\ln 6 = 1.791759$

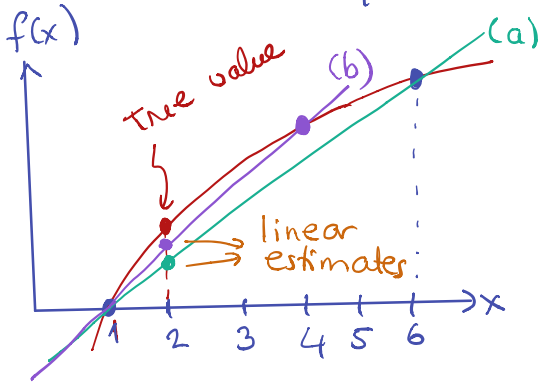
(b) " " $\ln 1$ and $\ln 4 = 1.386294$

(Note true value is $\ln 2 = 0.6931472$)

$$f_1(x) = f(x_1) + \frac{f(x_2) - f(x_1)}{x_2 - x_1}(x - x_1)$$

(a) $\ln 2 = \ln 1 + \dfrac{\ln 6 - \ln 1}{6 - 1}(2 - 1) = 0.358 \Rightarrow \varepsilon_t = 48.3\%$

(b) $\ln 2 = \ln 1 + \dfrac{\ln 4 - \ln 1}{4 - 1}(x - 1) = 0.4620 \Rightarrow \varepsilon_t = 33\%$



Quadratic Interpolation

If 3 data pts are available, we can fit a 2nd order polynomial

available pts:
$(x_1, f(x_1))$
$(x_2, f(x_2))$
$(x_3, f(x_3))$

(1) $$\boxed{f_2(x) = b_1 + b_2(x-x_1) + b_3(x-x_1)(x-x_2)}$$

when $x = x_1 \Rightarrow$ $\boxed{f(x_1) = b_1}$ (2)

when $x = x_2 \Rightarrow$ $f(x_2) = f(x_1) + b_2(x_2 - x_1)$

$$\boxed{b_2 = \frac{f(x_2) - f(x_1)}{(x_2 - x_1)}} \quad (3)$$

(slope of line connecting pts x_1 and x_2)

when $x = x_3 \Rightarrow$ $f(x_3) = \underbrace{f(x_1)}_{b_1} + \underbrace{\frac{f(x_2) - f(x_1)}{x_2 - x_1}}_{b_2}(x_3 - x_1) + b_3(x_3 - x_1)(x_3 - x_2)$

↓ solve for b_3

$$\Rightarrow \boxed{b_3 = \frac{\dfrac{f(x_3) - f(x_2)}{x_3 - x_2} - \dfrac{f(x_2) - f(x_1)}{x_2 - x_1}}{x_3 - x_1}} \quad (4)$$

↓ similar to the 2nd order derivative formula

* Eq (1) has a structure similar to Taylor expansion.

Ex Approximate $\ln 2$ using a 2nd order polynomial, 3 pts:

$x_1 = 1$, $f(x_1) = 0$

$x_2 = 4$, $f(x_2) = 1.386294$

$x_3 = 6$, $f(x_3) = 1.791759$

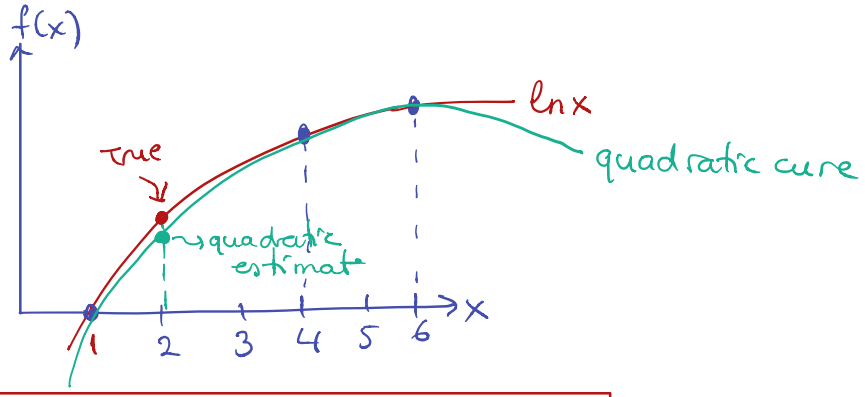
From (2) $b_1 = 0$

From (3) $b_2 = 0.462$

From (4) $b_3 = -0.0518$

$$f_2(x) = \underbrace{0.462098}_{b_2}(x-1) - \underbrace{0.0518}_{b_3}(x-1)(x-4)$$

$$f_2(2) = 0.5658 \quad \Rightarrow \quad \varepsilon_t = 18\%$$



Lagrange Interpolating Polynomial

Formulate the linear interpolating polynomial as the weighted average of two values:

Given: $(x_1, f(x_1))$, $(x_2, f(x_2))$

$$f(x) = L_1 f(x_1) + L_2 f(x_2) \qquad (1)$$

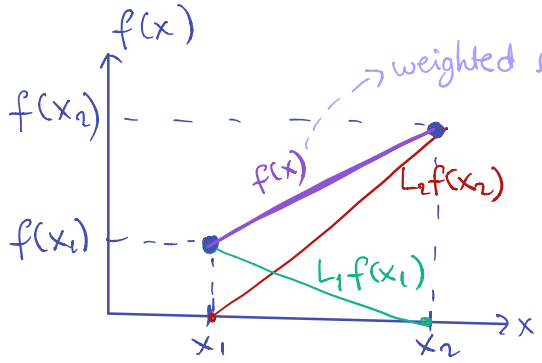
$$L_1 = \frac{x-x_2}{x_1-x_2} : \begin{cases} 1, \text{ at } x = x_1 \\ 0, \text{ at } x = x_2 \end{cases} \qquad (2)$$

$$L_2 = \frac{x-x_1}{x_2-x_1} : \begin{cases} 0, \text{ at } x_1 \\ 1, \text{ at } x_2 \end{cases} \qquad (3)$$

Put (2) and (3) into (1):

$$f(x) = \frac{x-x_2}{x_1-x_2} f(x_1) + \frac{x-x_1}{x_2-x_1} f(x_2)$$

Linear Lagrange interpolating polynomial



weighted sum of red line and green line

A parabola through three points:

Given: 3 pts: $(x_1, f(x_1))$, $(x_2, f(x_2))$, $(x_3, f(x_3))$
(pt 1, pt 2, pt 3)

Use 3 parabolas with each one passing thru one of the pts and is equal to zero at the other two.

2nd order Lagrange interpolating polynomial is:

$$f_2(x) = \underbrace{\frac{(x-x_2)(x-x_3)}{(x_1-x_2)(x_1-x_3)}}_{L_1(x)} f(x_1) + \underbrace{\frac{(x-x_1)(x-x_3)}{(x_2-x_1)(x_2-x_3)}}_{L_2(x)} f(x_2) + \underbrace{\frac{(x-x_1)(x-x_2)}{(x_3-x_1)(x_3-x_2)}}_{L_3(x)} f(x_3)$$

$L_1(x)$: zero at x_2 and x_3, one at x_1

$L_2(x)$: zero at x_1, x_3, one at x_2

$L_3(x)$: zero at x_1, x_2, one at x_3

General formula:

$$f_{n-1}(x) = \sum_{i=1}^{n} L_i(x) f(x_i) \quad \text{where} \quad L_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

EX Given the measurements

$x_1 = 0$, $f(x_1) = 3.85$

$x_2 = 20$, $f(x_2) = 0.8$

$x_3 = 40$, $f(x_3) = 0.212$

Evaluate the function at $T = 15^\circ C$.

Use Lagrangian interpolation: a) First order polynomial
b) 2nd " "

} Self-study exercise

Soln First order polynomial

$$f_1(x) = \frac{(x-x_2)}{(x_1-x_2)} f(x_1) + \frac{(x-x_1)}{(x_2-x_1)} f(x_2)$$

$$f_1(x) = \frac{(x-20)}{(0-20)}(3.85) + \frac{(x-0)}{(20-0)}(0.8)$$

$$f_1(15) = 1.5625$$

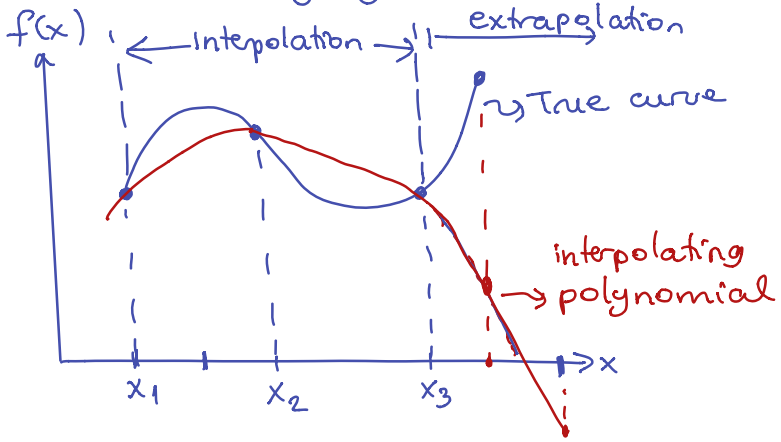
Second order polynomial:

$$f_2(x) = \frac{(x-x_2)(x-x_3)}{(x_1-x_2)(x_1-x_3)} f(x_1) + \frac{(x-x_1)(x-x_3)}{(x_2-x_1)(x_2-x_3)} f(x_2) + \frac{(x-x_1)(x-x_2)}{(x_3-x_1)(x_3-x_2)} f(x_3)$$

$$f_2(x) = \frac{(x-20)(x-40)}{(0-20)(0-40)}(3.85) + \frac{(x-0)(x-40)}{(20-0)(20-40)}(0.8) + \frac{(x-0)(x-20)}{(40-0)(40-20)} 0.212$$

$$f_2(15) = 1.3316$$

Extrapolation: process of estimating $f(x)$ at an x value outside the range of known base pts: $x_1, x_2, \dots x_n$.

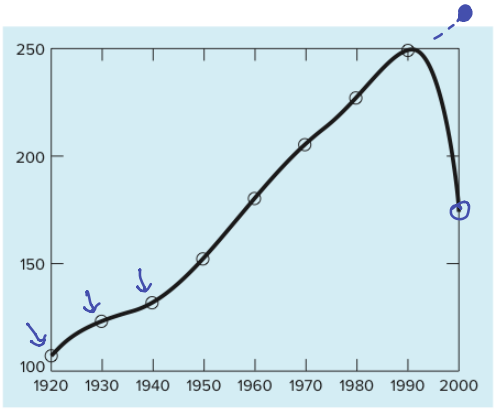


* Extrapolation must be done with extreme case.

Oscillations: higher-order polynomials cause oscillation.
very sensitive to roun-off error.

EXTRAPOLATION EXAMPLES: Dangers of Higher Order Interpolation

Ex

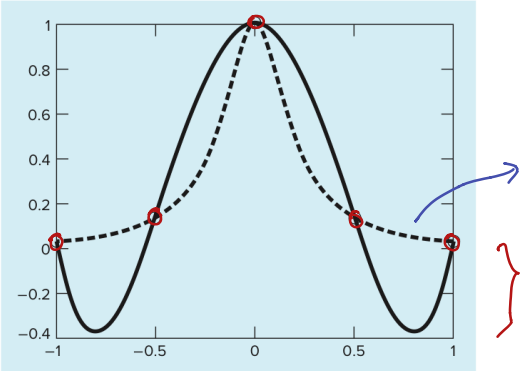


The 7th order polynomial fits the data between 1920–1990 very well, but gives wrong predictions for 2000.

FIGURE 17.11
Use of a seventh-order polynomial to make a prediction of U.S. population in 2000 based on data from 1920 through 1990.

Ex

FIGURE 17.12
Comparison of Runge's function (dashed line) with a fourth-order polynomial fit to 5 points sampled from the function.



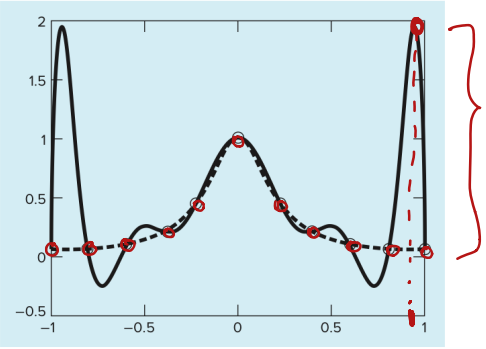


FIGURE 17.13
Comparison of Runge's function (dashed line) with a tenth-order polynomial fit to 11 points sampled from the function.

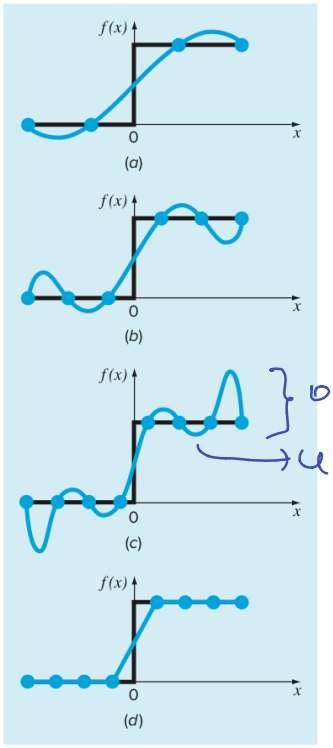
Higher-order interpolation should be avoided unless absolutely necessary.

Chapter 18 - Spliner and Piecewise Interpolation

- Spliner minimize oscillations by fitting lower-order polynomials to data in a piecewise fashion.

Previous chapter: to interpolate n data points $(n-1)^{th}$ order polynomials were used ⇒ may lead to roundoff errors and oscillations.

Alternative approach: use lower-order polynomials in a piecewise way, using splines.



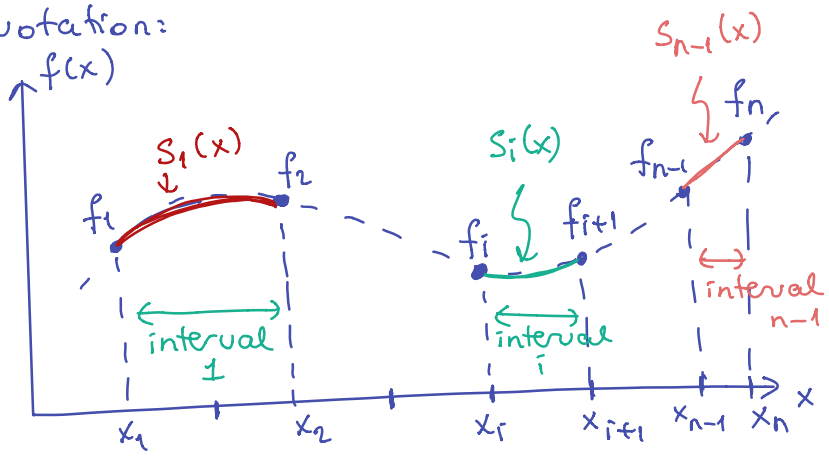
overshoot

undershoot

FIGURE 18.1
A visual representation of a situation where splines are superior to higher-order interpolating polynomials. The function to be fit undergoes an abrupt increase at $x = 0$. Parts (*a*) through (*c*) indicate that the abrupt change induces oscillations in interpolating polynomials. In contrast, because it is limited to straight-line connections, a linear spline (*d*) provides a much more acceptable approximation.

Linear Splines

Notation:



n data pts
$(i = 1, \ldots, n)$

n-1 intervals

Each interval has its own spline function: $S_i(x)$

Linear splines: each func. is a straight line

$$S_i(x) = a_i + b_i (x - x_i)$$

left pt: (x_i, f_i)

right pt: (x_{i+1}, f_{i+1})

$$a_i = f_i$$

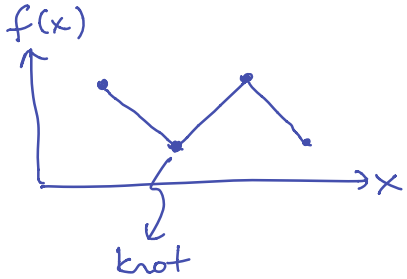
$$b_i = \frac{f_{i+1} - f_i}{x_{i+1} - x_i}$$

$$S_i(x) = f_i + \frac{f_{i+1} - f_i}{x_{i+1} - x_i}(x - x_i)$$

→ Newton's 1st order poly.

For new x

1) Locate the interval within which x lies
2) Use the appropriate eqn to evaluate spline at x.



Disadvantages of 1st order splines:

- They're not smooth at the knots (where splines meet), slope change abruptly.
  $\Rightarrow$ 1st derivative of the spline is discontinuous.

Soln: use higher-order polynomial splines, ensure smoothness at the knots.

Quadratic splines: have continuous 1st derivatives at the knots.

objective: derive a 2nd order polynomial for each interval btwn data ptr.

for each interval:

$$s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2$$

a_i, b_i, c_i → unknown

for n ptr $\Rightarrow$ (n-1) intervals $\Rightarrow$ 3(n-1) unknowns.

→ continuity condition

→ Function values equal at knots $s_i(x_{i+1}) = s_{i+1}(x_{i+1})$

→ First derivatives " " $s_i'(x_{i+1}) = s_{i+1}'(x_{i+1})$

Cubic Splines:

- Most used in practice
- objective: derive a 3rd order polynomial for each interval

$$s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

Derivation: Estimate unknown for each interval

- continuity
- First derivatives equal at knots
- 2nd " " " interior knots.

EXAMPLE : Interpolation with Spliner

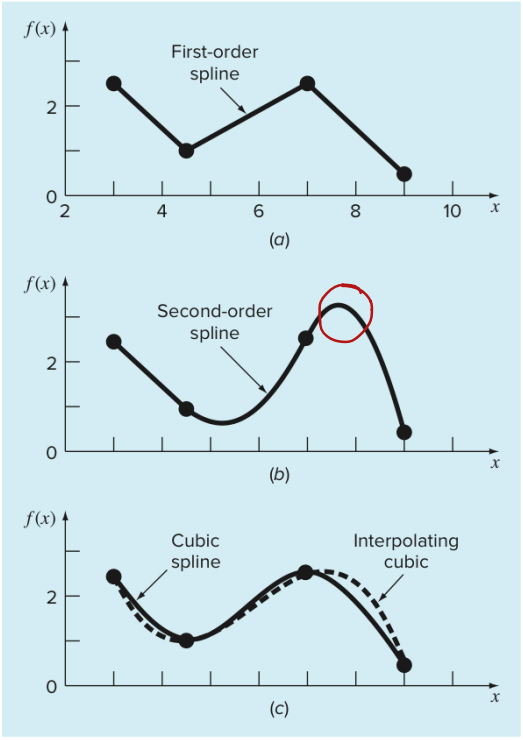
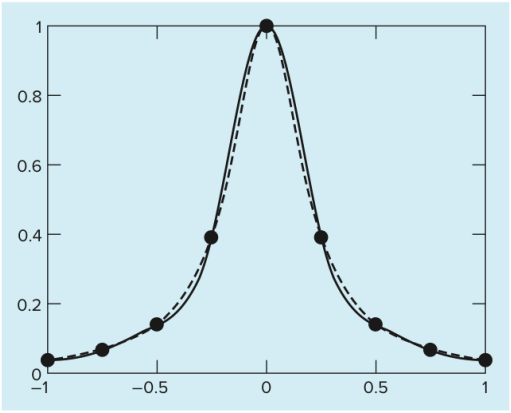


FIGURE 18.4
Spline fits of a set of four points. (*a*) Linear spline, (*b*) quadratic spline, and (*c*) cubic spline, with a cubic interpolating polynomial also plotted.

FIGURE 18.6
Comparison of Runge's function (dashed line) with a 9-point not-a-knot spline fit generated with MATLAB (solid line).



Piecewise Interpolation in MATLAB

Built-in functions:

- spline: cubic spline interpolation
- pchip: piecewise cubic Hermite interpolation
- interp1: spline and Hermite interpolation, other interpolations

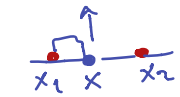
```
>> yy = spline(x,y,xx)
```

xx → new x value to evaluate

x, y → vectors containing the values to be interpolated

```
>> yi = interp1(x,y,xi,'method')
```

x, y → values to be interpolated



- 'nearest': nearest neighbor interpolation (zero-order polynomials)
- 'linear': linear interpolation (straight liner to connect the pts.)
- 'spline': piecewise cubic spline interpolation (identical to spline function)
- 'pchip' and 'cubic': piecewise cubic Hermite interpolation.

(Default is 'linear')

Cubic Hermite:

- continuous first derivatives
- second derivatives are not necessarily continuous
- interpolation is "shape preserving"
- interpolated values do not tend to overshoot the data points as can sometimes happen in cubic splines.

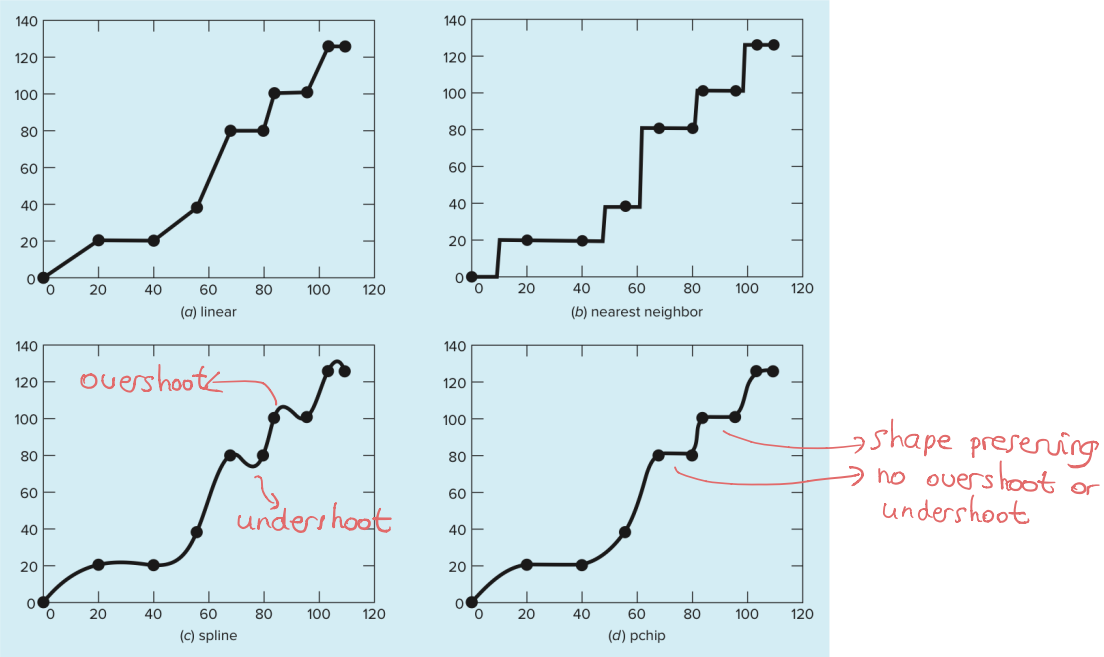


FIGURE 18.8
Use of several options of the `interp1` function to perform piecewise polynomial interpolation on a velocity time series for an automobile.